

Übungsblatt 5

Programmierung von Web-Systemen in Java

5430UE Web and Data Engineering

24. März 2026

Prof. Dr. Michael Granitzer

Lernziele

- Die Komponenten des **Java Web Stack** (Servlet-Container, Build-Tools, Test-Frameworks) einordnen
- **Servlets**, **JSP** und **JavaBeans** als Grundbausteine kennen und einsetzen
- **Spring Boot** für REST-APIs und **Spring Data / JPA** für Persistenz anwenden
- Den **Spring Request-Flow** (DispatcherServlet → Controller → Service → Repository) verstehen
- **Dependency Injection** und die **Test-Pyramide** (Unit / Integration / E2E) anwenden

Modul A — Java Web Stack

Übung 5.A.1: Java Web Stack Begriffe

Ordnen Sie die folgenden Technologien der richtigen Kategorie im Java Web Stack zu und geben Sie eine kurze Beschreibung ihrer Aufgabe:

Technologie	Kategorie	Kurzbeschreibung
Thymeleaf	?	?
Tomcat	?	?
Maven	?	?
JUnit	?	?
Jenkins	?	?
H2	?	?
Spring Framework	?	?

Mögliche Kategorien: View Template Engine, Application Server, Build Management Tool, Test Framework, CI/CD Tool, Datenbank, Web Framework

Übung 5.A.2: Maven Grundlagen

1. Was ist ein Maven-Artefakt? Wie ist eine Maven-Koordinate aufgebaut? Geben Sie ein Beispiel.
2. Was bedeutet semantische Versionierung (Semantic Versioning)? Erklären Sie MAJOR.MINOR.PATCH anhand eines Beispiels.
3. Nennen Sie mindestens vier Phasen des Maven Default Lifecycle in der richtigen Reihenfolge.

Modul B — Servlets

Übung 5.B.1: Servlet Lifecycle

Ordnen Sie die folgenden Code-Schnipsel dem richtigen Schritt im Servlet-Lebenszyklus zu. Beschreiben Sie außerdem, was diesen Schritt auslöst.

```
// Schnipsel A
protected void doGet(HttpServletRequest req, HttpServletResponse res)
    throws ServletException, IOException {
    res.setContentType("text/html");
    PrintWriter out = res.getWriter();
    out.println("<h1>Hello " + req.getParameter("name") + "</h1>");
}

// Schnipsel B
public void init(ServletConfig config) throws ServletException {
    super.init(config);
    // einmalige Initialisierung
}

// Schnipsel C
public void destroy() {
    // Ressourcen freigeben
}

// Schnipsel D
@WebServlet(urlPatterns = {"/hello"})
```

Beantworten Sie außerdem: Welche Methode ruft der Container auf, wenn eine HTTP-POST-Anfrage eintrifft?



Übung 5.B.2: Session Tracking Vergleich

Vergleichen Sie die fünf gängigen Session-Tracking-Methoden. Füllen Sie die Tabelle aus:

Methode	Funktioniert ohne Cookies?	XSS-Risiko?	Typischer Einsatz
URL-Kodierung (URL Rewriting)	?	?	?
Hidden Fields	?	?	?
Cookies	?	?	?
HTTP Authorization Header	?	?	?
Session Tracking API (HttpSession)	?	?	?

Welche Methode nutzt Spring Boot's HttpSession standardmäßig?

Übung 5.B.3: JSP und Java Beans

Ein Formular auf `form.html` sendet eine POST-Anfrage an `/convert` mit dem Parameter `amount`. Das Servlet soll den Betrag von EUR in USD umrechnen (Kurs: 1 EUR = 1.10 USD) und das Ergebnis über se

eine JSP (`result.jsp`) ausgeben.

1. Skizzieren Sie den Request-Verarbeitungsfluss: Formular → Servlet → JSP.
2. Wie übergibt das Servlet das Ergebnis an die JSP, ohne es in der URL zu kodieren?
3. Welche Methode des Servlets (`doGet` oder `doPost`) wird aufgerufen, und warum?
4. Was ist ein **Java Bean** im JSP-Kontext, und welche Voraussetzungen muss eine Klasse erfüllen?

Übung 5.B.4: CurrencyServlet — eigene REST-API

In Übungsblatt 4 haben Sie einen Währungsrechner gegen die `exchangerate.host`-API gebaut. Implementieren Sie nun eine **eigene** Servlet-basierte API, die das gleiche Format liefert:

- GET `/currency` → JSON mit Wechselkursen von **EUR** in Fremdwährungen (USD, CHF)
- GET `/currency?base=USD` → Wechselkurse von **USD** in Fremdwährungen
- Wechselkurse dürfen im Quelltext fest verdrahtet werden

Implementieren Sie die `doGet()`-Methode des `CurrencyServlet` mit `@WebServlet`-Annotation. Welche Bibliothek verwenden Sie für die JSON-Serialisierung, und warum sollten Sie nicht von `String.format` benutzen?

Übung 5.B.5: JSP-Frontend mit FinanceBean

Erweitern Sie das CurrencyServlet aus Übung 5.B.4 um eine doPost()-Methode, die Formular-Daten von einer HTML-Seite entgegennimmt und das Ergebnis über eine JSP rendert.

Gegeben:

```
<!-- index.html -->
<form action="currency" method="POST">
  <input name="value" type="number" value="100"/>
  <select name="to">
    <option value="USD">USD</option>
    <option value="EUR">EUR</option>
  </select>
  <button type="submit">Umrechnen</button>
</form>
```

```
<!-- solution.jsp -->
<%@ page import="de.unipassau.currency.FinanceBean" %>
<% FinanceBean result = (FinanceBean) request.getAttribute("result"); %>
<h1>Ergebnis: <%= result.getAmount() %> <%= result.getCurrency() %></h1>
```

1. Implementieren Sie die FinanceBean-Klasse.
2. Implementieren Sie doPost(): liest value und to, rechnet um, befüllt die Bean, hängt sie als Request-Attribut result an und leitet auf solution.jsp weiter.

Modul C — Spring Boot & REST

Übung 5.C.1: Spring Boot REST-Controller

Implementieren Sie einen Spring Boot REST-Controller für ein Studierenden-Verwaltungssystem:

- GET /students — alle Studierenden zurückgeben
- GET /students/{id} — Studierendem mit ID; 404 wenn nicht vorhanden
- POST /students — neuen Studierenden anlegen; 201 mit Location-Header
- DELETE /students/{id} — Studierenden löschen; 204 bei Erfolg

Verwenden Sie eine StudentRepository-Schnittstelle. Welche Spring-Annotationen benötigen Sie?

Übung 5.C.2: Dependency Injection und Testbarkeit

Vergleichen Sie folgende zwei Implementierungen:

```
// Version A
@Service
public class OrderService {
    private final EmailService email = new EmailService();
    public void placeOrder(Order o) { email.send(o); }
}
```

```
// Version B
@Service
public class OrderService {
```



```
private final EmailService email;
public OrderService(EmailService email) { this.email = email; }
public void placeOrder(Order o) { email.send(o); }
}
```

1. Welche Version ist testbarer? Warum?
2. Was macht Spring automatisch bei Version B?
3. Schreiben Sie einen Unit-Test für Version B, der EmailService mockt.

Übung 5.C.3: Test-Pyramide

Beschreiben Sie die drei Ebenen der Test-Pyramide für eine Spring Boot Anwendung:

1. Was testet jede Ebene? Nennen Sie ein konkretes Beispiel.
2. Wie verhält sich die empfohlene Anzahl der Tests von der untersten zur obersten Ebene?
3. Welche Spring-Test-Annotationen oder -Bibliotheken sind typisch?

Übung 5.C.4: MVC, MVP und MVVM im Vergleich

1. Füllen Sie die Tabelle aus.
2. Wann wählt man Spring MVC (Thymeleaf) und wann React + Spring REST?
3. Warum ist MVP besonders testfreundlich?

Übung 5.C.5: Spring CommandLineRunner

Bei der Entwicklung einer Spring Boot-Anwendung sollen beim Start automatisch Testdaten in die Datenbank geladen werden.

1. Was ist ein `CommandLineRunner` in Spring Boot, und wann wird er ausgeführt?
2. Implementieren Sie einen `CommandLineRunner`, der beim Start 3 Produkte in ein `ProductRepository` (JPA) lädt.
3. Was ist der Unterschied zwischen `CommandLineRunner` und `ApplicationRunner`?

Modul D — Persistenz mit JPA

Übung 5.D.1: Entity-Mapping und Repository

Modellieren Sie eine JPA-Entity `Course` mit folgenden Attributen:

- `id` (automatisch generiert)
- `title` (nicht null, max. 200 Zeichen)
- `credits` (Ganzzahl, 1–10)
- `instructor` (String)

Erstellen Sie dazu ein `CourseRepository` mit einer Methode, die alle Kurse eines bestimmten Dozenten findet.

Welche JPA-Annotationen und welche Spring Data-Schnittstelle verwenden Sie?

Übung 5.D.2: Kursanmeldung mit Voraussetzungsprüfung

Implementieren Sie `EnrollmentService.enroll(studentId, courseId)` mit folgenden Regeln:



1. Student muss existieren
2. Kurs muss existieren
3. Student darf noch nicht eingeschrieben sein
4. Student muss alle Vorkurskurse abgeschlossen haben

Welchen HTTP-Status-Code soll der Controller für jede Exception zurückgeben?

Übung 5.D.3: JPA Vererbungsstrategien

Sie modellieren eine Klassenhierarchie: Person, Student und Lehrkraft.

Skizzieren Sie, wie die drei JPA-Vererbungsstrategien diese Hierarchie in Datenbanktabellen abbilden würden:

1. SINGLE_TABLE
2. JOINED
3. TABLE_PER_CLASS

Welche Strategie würden Sie für dieses Szenario wählen und warum?

Materialien zum Übungsblatt

Zur Übung mit Servlets, JSP und Spring Boot stehen vollständige Code-Templates bereit:

Servlets / JSP

- [servlets-template/](#) — minimales Servlet-Template (CurrencyServlet.java, FinanceBean.java, index.html, api.html, TestCurrencyServlet.java) zum selbst Ausfüllen
- [servlets-filled/](#) — funktionierende Referenzlösung

Spring Boot REST

- [spring-boot-rest/](#) — vollständige Studenten-Service-Anwendung mit StudentController, StudentService, Student, Course, DTOs, Initialisierung und Tests

*Alle Materialien finden Sie auch zentral auf der **Kursseite** unter Materials → Download.*